Setting up the online server

For online trading and battles (plan 07). Written 12 October 2026. About an hour the first time, and about $5 to $6 a month after that.

What you end up with. One small Linux server on the internet, with a free web address such as `yourname.duckdns.org`. Players' games connect to it over an encrypted connection (`wss://`). The server program runs as a service that restarts on its own, the operating system installs its security updates by itself, and the database is backed up every night.

When to do what. You can do steps 1 to 6 today. Steps 7 to 10 need the server program, which plan 07's session O4 builds. Until then the server sits ready and costs the monthly fee.

1. Which provider

The game's server is tiny. A trade or a battle turn is a few hundred bytes, and a battle takes a few milliseconds of work. Any of the providers below can serve dozens of friends at once. The table shows **prices as listed in October 2026**. Several providers raised their prices in 2026, so check the price on the order page before you buy.

Provider and plan	CPU / memory / disk	Traffic a month	Price a month	Notes
Hetzner Cloud CX23 (recommended)	2 vCPU / 4 GB / 40 GB	20 TB	about €5 to €6, including the IPv4 address	The most server for the money. Data centres in Germany and Finland (Singapore and the US too). A European company, so GDPR is simple. It sometimes asks for ID when you sign up.
Hetzner Cloud CAX11 (ARM)	2 vCPU / 4 GB / 40 GB	20 TB	about €4 to €5	Cheaper. .NET runs on ARM, so publish with `-r linux-arm64` in step 7. Some locations run out of these.
Vultr Cloud Compute 1 GB	1 vCPU / 1 GB / 25 GB	about 1 TB	$5	Exactly $5, with 32 locations (Frankfurt, Warsaw, Amsterdam and others). 1 GB of memory is enough for this server.
Akamai (Linode) Nanode 1 GB	1 vCPU / 1 GB / 25 GB	1 TB	$5	Also exactly $5, with free DDoS protection.
DigitalOcean Basic 1 GB	1 vCPU / 1 GB / 25 GB	1 TB	$6	The best documentation, but the most expensive of the five.

Recommendation: Hetzner Cloud CX23, in the location closest to most of your players. It has four times the memory of the $5 plans for about the same money. If the budget has to stay at exactly $5, take **Vultr's $5 plan**. The steps below are written for Hetzner. Vultr and Akamai have the same choices under different names, which section 4 lists.

2. Before you start

- A payment card, and an e-mail address for the provider account.
- Your own computer with a terminal. On Windows, PowerShell has `ssh` and `scp` built in, so you don't need to install anything.
- A name for the server that doesn't contain "Pokémon" or any other Nintendo name, for example `sinnoh-link` or `yourname-games`. Plan 07 explains why: a fan project's online service should not draw attention to itself.

> **Keep two things private:** the SSH key file on your computer (`id_ed25519` , the one *without* `.pub`), and the server's secret from step 7. Never put either one in the repository.

3. The steps

1 Make an SSH key on your computer

The key is how you log in to the server, in place of a password. In PowerShell, or in a terminal on macOS or Linux:

```
ssh-keygen -t ed25519 -C "game server"
# press Enter to keep the default file; choose a passphrase you will remember
cat ~/.ssh/id_ed25519.pub       # on Windows: type $env:USERPROFILE\.ssh\id_ed25519.pub
```

Copy the line it prints. It starts with `ssh-ed25519` . That is the public half, and it is safe to paste into the provider's website.

2 Create the server (Hetzner)

1. Sign up at **console.hetzner.com**, confirm your e-mail and add a payment method.
2. Create a **project** (for example "Games") and open it. Choose **Add Server**.
3. **Location**: the one closest to most players (Falkenstein or Nuremberg in Germany, or Helsinki).
4. **Image**: **Ubuntu**, the newest LTS version offered (26.04 or 24.04).
5. **Type**: Shared vCPU, **x86**, **CX23** (or Arm64, **CAX11**).
6. **Networking**: keep **Public IPv4** turned on. The game needs it, as step 3 explains. IPv6 can stay on.
7. **SSH keys**: *Add SSH key*, paste the line from step 1, and give it a name.
8. **Firewalls**: *Create Firewall* with three inbound rules, all TCP, from any IPv4 and IPv6 address: port **22** (SSH), port **80** (HTTP, which Caddy needs to get its certificate) and port **443** (HTTPS, where the game connects). Apply the firewall to this server.
9. **Backups**: optional. They cost 20% of the server price, and step 10 sets up a backup of its own anyway.
10. **Name**: for example `sinnoh-link` . Choose **Create & Buy now**.

After a few seconds the server shows its **IPv4 address**, for example `203.0.113.25` . Write it down.

3 Give it a free name with DuckDNS

HTTPS certificates are issued to names, not to bare IP addresses, so the server needs a name. DuckDNS gives you one for free.

1. Go to **www.duckdns.org** and sign in (with GitHub, Google or another account).
2. Type a subdomain, for example `sinnoh-link`, and choose **add domain**.
3. In the **current ip** box next to it, type the server's **IPv4** address and choose **update ip**.
4. Leave the **IPv6** box empty. The game binds each link code to the players' IP addresses, and home IPv6 addresses change by themselves every day, so the server must be reached over IPv4 only.

Your server's address is now `sinnoh-link.duckdns.org`. Hetzner's IP addresses don't change, so DuckDNS never needs updating. If you own a domain already, add an **A** record for a subdomain instead, with no AAAA record.

4 Log in and lock it down

```
ssh root@203.0.113.25           # your IPv4; answer "yes" the first time

apt update && apt full-upgrade -y
adduser admin                   # your everyday login; choose a password (it's for sudo only)
usermod -aG sudo admin
rsync --archive --chown=admin:admin ~/.ssh /home/admin
```

Now turn off password logins and logins as root:

```
cat > /etc/ssh/sshd_config.d/10-hardening.conf <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
EOF
systemctl restart ssh
```

Before you close this window, open a second terminal and check that `ssh admin@203.0.113.25` works. If it does, the rest of the steps run as `admin`, with `sudo` in front.

A second firewall on the server itself, in case the provider's firewall is ever removed:

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Automatic security updates, with a restart at 4 a.m. when an update needs one:

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades      # answer Yes
echo 'Unattended-Upgrade::Automatic-Reboot "true";
Unattended-Upgrade::Automatic-Reboot-Time "04:00";' | sudo tee /etc/apt/apt.conf.d/52auto-rebo
ot
```

5 Install Caddy (HTTPS in front of the game server)

Caddy gets a free certificate from Let's Encrypt and renews it by itself. It passes the game's WebSocket connections through to the server program, along with the player's real IP address.

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' \
  | sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' \
  | sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt install -y caddy
```

Replace the whole of `/etc/caddy/Caddyfile` with this (use `sudo nano /etc/caddy/Caddyfile`):

```
sinnoh-link.duckdns.org {
    reverse_proxy 127.0.0.1:5080
}
```

```
sudo systemctl reload caddy
curl -I https://sinnoh-link.duckdns.org     # "502 Bad Gateway" is right for now: HTTPS works,
nothing behind it yet
```

6 A user and folders for the game server

```
sudo useradd --system --home /var/lib/linkserver --create-home --shell /usr/sbin/nologin links
erver
sudo mkdir -p /opt/linkserver
sudo apt install -y sqlite3           # for backups and for looking inside the database
```

Steps 1 to 6 are everything you can do before the server program exists.

7 Put the server program on it (once plan 07 · O4 exists)

On your computer, in the repository folder, build a self-contained copy, so the server doesn't need .NET installed:

```
dotnet publish PokemonPlatinumServer -c Release -r linux-x64 --self-contained -o publish/serve
r
# for a CAX11 (ARM) server: -r linux-arm64
scp -r publish/server admin@203.0.113.25:/tmp/linkserver
```

On the server:

```
sudo rsync -a --delete /tmp/linkserver/ /opt/linkserver/
sudo chmod +x /opt/linkserver/PokemonPlatinumServer

# the secret that hashes players' IP addresses; made once, never changed, never shared
echo "LinkServer__IpHashSecret=$(openssl rand -hex 32)" | sudo tee /etc/linkserver.env
sudo chmod 600 /etc/linkserver.env
```

A service, so the program starts with the machine and restarts if it stops. Create `/etc/systemd/system/linkserver.service`:

```
[Unit]
Description=Game link server
After=network-online.target

[Service]
User=linkserver
WorkingDirectory=/var/lib/linkserver
ExecStart=/opt/linkserver/PokemonPlatinumServer
EnvironmentFile=/etc/linkserver.env
Environment=ASPNETCORE_URLS=http://127.0.0.1:5080
Environment=LinkServer__Database=/var/lib/linkserver/link.db
Environment=LinkServer__TrustedProxy=127.0.0.1
Restart=always
RestartSec=5
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/linkserver

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now linkserver
systemctl status linkserver             # should say "active (running)"
```

The exact setting names (`LinkServer__…`) and the program's name are fixed when O4 writes the server. O7 ships this file and the Caddyfile in the repository, so you copy them rather than type them.

8 Check it works

```
curl https://sinnoh-link.duckdns.org/health     # O4's health check: "ok" and the protocol ver
sion
journalctl -u linkserver -f                     # the server's log, live; Ctrl+C to stop
```

In the log, a player's first connection should show their real IP address being hashed, not `127.0.0.1`. If every player shows `127.0.0.1`, they would all get the same friend code, and section 5 has the fix.

9 Point the game at it

In each player's `settings.json` (O7 adds this to the options screen too):

```
"OnlineServer": "wss://sinnoh-link.duckdns.org/link"
```

Then one player chooses **Make a code** at the Global Link counter upstairs in any Pokémon Center, and the other types the code.

10 Nightly backup of the database

```
sudo mkdir -p /var/backups/linkserver
echo '30 3 * * * root sqlite3 /var/lib/linkserver/link.db ".backup /var/backups/linkserver/lin
k-$(date +\%F).db" && find /var/backups/linkserver -name "link-*.db" -mtime +14 -delete' \
  | sudo tee /etc/cron.d/linkserver-backup
```

This keeps fourteen days of copies on the server. Now and then, keep one on your own computer too:

```
scp admin@203.0.113.25:/var/backups/linkserver/link-*.db ./backups/
```

4. The same on Vultr or Akamai

Hetzner's name	Vultr	Akamai (Linode)
Add Server	Deploy, then Cloud Compute (Shared CPU)	Create Linode
CX23 / CAX11	the $5 plan, 1 vCPU and 1 GB	Nanode 1 GB
Image: Ubuntu LTS	Operating system: Ubuntu LTS	Image: Ubuntu LTS
SSH keys	SSH Keys	SSH Keys (Authorized Keys)
Firewalls	Firewall group with TCP 22, 80, 443	Cloud Firewall with TCP 22, 80, 443

From step 3 on, everything is the same.

5. When something goes wrong

What you see	Why, and what to do
`ssh` hangs or is refused	Port 22 isn't open in the provider's firewall, or the firewall isn't applied to the server. If you locked yourself out, the provider's web console (Hetzner: the server's *Console* tab) still logs you in.
Caddy can't get a certificate	DuckDNS doesn't point at this IP yet, or port 80 is closed. Check with `journalctl -u caddy`, fix it, then run `sudo systemctl reload caddy`.
Every player gets the same friend code	The server is reading Caddy's address instead of the player's. The `TrustedProxy` setting in step 7 must be `127.0.0.1`, and the server must use ASP.NET Core's forwarded-headers middleware (O4 does).
A player's friend code changed	Their game lost its install token (a new computer, or a deleted `settings.json`) *and* their IP changed. Nothing is lost except the code.
A player can't connect, but others can	They might reach the internet only over IPv6, which some mobile networks do. Plan 07 accepts that for now.
Disk full	Run `sudo journalctl --vacuum-size=200M` and check `/var/backups/linkserver`.

6. Updating, costs, and stopping

- **A new server version**: repeat the `publish`, `scp` and `rsync` lines of step 7, then run `sudo systemctl restart linkserver`. Connected games drop and have to reconnect. A trade in progress finishes or rolls back through its receipt, but a battle in progress is lost (until plan 28 · I5 lets a battle outlive a restart), so update when nobody is playing.

- **Once a month**: log in, run `sudo apt update && sudo apt full-upgrade` (the automatic updates install only security fixes), and look at `df -h` and `systemctl status linkserver`.
- **Costs**: the plan's monthly price, billed by the hour, with a cap. Traffic is far below every plan's allowance.
- **Stopping**: in the provider's console, delete the server. A stopped server is still billed, a deleted one isn't. Download a backup first if you might start again.

Sources: the providers' price pages and comparisons published in September and October 2026. Hetzner's prices changed twice in 2026, so check on the order page. Caddy's install commands are from caddyserver.com/docs/install.